TALLER PRÁCTICO · LABORATORIO LOCAL

Desarrollo Guiado por Pruebas (TDD) con JavaScript

Construye la lógica de un carrito de compras prueba por prueba, solo con Node.js y `node:test`.

ROJO → VERDE → REFACTOR

```
$ npm test
✔ un carrito nuevo no tiene ítems
✔ el cupón DESC10 descuenta el 10 %
✔ el IVA se calcula sobre el valor con descuento
ℹ tests 17  ℹ pass 17  ℹ fail 0
```

Repositorio del laboratorio
`github.com/erozog125/Laboratorio_TDD`

Prof. Edwin Rozo Gómez
Guía del estudiante

Antes de empezar

En este taller vas a construir, paso a paso y guiado por pruebas, la lógica de un **carrito de compras** como el de cualquier tienda en línea: agregar productos, eliminarlos, aplicar cupones de descuento y calcular el IVA. Todo con **JavaScript puro** y el ejecutor de pruebas que ya trae Node.js, sin instalar ninguna librería.

Objetivos de aprendizaje

- ✔ Explicar qué es TDD y por qué se escribe la prueba antes que el código.
- ✔ Aplicar el ciclo **Rojo → Verde → Refactor** en iteraciones pequeñas.
- ✔ Escribir pruebas unitarias con `node:test` y `node:assert`.
- ✔ Estructurar una prueba con el patrón **Preparar – Actuar – Comprobar**.
- ✔ Refactorizar con confianza apoyándote en las pruebas.

Requisitos

Necesitas	Cómo verificarlo
Node.js 20 o superior (recomendado 22 LTS)	`node -v` debe mostrar `v20` o un número mayor
Git	`git --version` debe mostrar una versión
Visual Studio Code (u otro editor)	Que puedas abrir una carpeta completa
Conocimientos básicos de JavaScript	Funciones, clases, arreglos y `reduce`

> **Dinámica sugerida: programación en parejas "ping-pong"**
> Trabajen en parejas. Una persona escribe la prueba que falla (rojo) y le pasa el teclado a la otra, que escribe el código para que pase (verde). Revisan juntos y en la siguiente iteración se invierten los roles.

1. Conceptos clave

¿Qué es TDD?

TDD (Test-Driven Development), o *desarrollo guiado por pruebas*, es una forma de programar en la que **primero escribes una prueba automática** que describe lo que el código debe hacer, y solo después escribes el código que la hace pasar. La prueba funciona como una especificación ejecutable: dice qué se espera y avisa de inmediato si algo se rompe.

El ciclo Rojo → Verde → Refactor

1 · ROJO
Escribe **una** prueba pequeña para un comportamiento que aún no existe. Ejecútala y **confirma que falla**.

→

2 · VERDE
Escribe el **código mínimo** que hace pasar la prueba. Aún no importa que quede elegante.

→

3 · REFACTOR
Limpia el código (nombres, duplicados, estructura) **con todas las pruebas en verde**.

Y vuelta a empezar con el siguiente comportamiento.

Reglas de oro

1. **No escribas código de producción sin una prueba que falle.**
2. **Escribe solo la prueba necesaria para fallar.** Una iteración a la vez.
3. **Escribe solo el código necesario para pasar.** Ni una línea más.
4. **Siempre mira la prueba en rojo.** Si nunca la viste fallar, no sabes si está probando algo.
5. **Refactoriza solo en verde.**

Anatomía de una prueba: Preparar – Actuar – Comprobar

Toda prueba unitaria tiene tres partes. En inglés se conoce como **AAA** (*Arrange, Act, Assert*):

```
test('agregar un producto actualiza el subtotal', () => {
  // Preparar (Arrange)
  const carrito = new Carrito();

  // Actuar (Act)
  carrito.agregar('Mouse', 50000, 2);

  // Comprobar (Assert)
  assert.equal(carrito.subtotal(), 100000);
});
```

Las herramientas: `node:test` y `node:assert`

Node.js trae su propio ejecutor de pruebas. No hay que instalar Jest, Mocha ni nada más.

Función	Para qué sirve
`describe(nombre, fn)`	Agrupa pruebas relacionadas.
`test(nombre, fn)`	Define una prueba individual.
`beforeEach(fn)`	Se ejecuta antes de **cada** prueba; ideal para crear un objeto limpio.
`assert.equal(real, esperado)`	Comprueba que dos valores sean estrictamente iguales (`===`).
`assert.deepEqual(a, b)`	Compara objetos o arreglos por su contenido.
`assert.throws(fn, /texto/)`	Comprueba que una función lance un error cuyo mensaje contenga ese texto.

Cómo leer la terminal

Símbolo	Significado
✔	La prueba pasó.
✖	La prueba falló. Al final aparece la sección `failing tests` con el motivo.
▶	Inicio de un grupo (`describe`).
`ℹ tests` / `ℹ pass` / `ℹ fail`	Total de pruebas, cuántas pasaron y cuántas fallaron. **Estos tres números son tu punto de control en cada paso.**

2. Preparar el laboratorio

El proyecto inicial ya está listo en un repositorio. Solo tienes que clonarlo y comprobar que funciona.

```
github.com/erozog125/Laboratorio_TDD
```

Paso 1. Verifica tus herramientas

Abre una terminal (PowerShell, CMD o Git Bash) y ejecuta:

```
node -v
git --version
```

Resultado esperado: dos líneas con números de versión, por ejemplo `v22.x.x` y `git version 2.x.x`. Si `node -v` muestra una versión menor a `v20`, actualiza Node.js antes de continuar.

Paso 2. Clona el repositorio

Ubícate en la carpeta donde guardas tus proyectos y ejecuta:

```
git clone https://github.com/erozog125/Laboratorio_TDD.git
cd Laboratorio_TDD
```

Resultado esperado: Git muestra `Cloning into 'Laboratorio_TDD'...` y la terminal queda ubicada dentro de la carpeta `Laboratorio_TDD`.

Paso 3. Abre el proyecto en VS Code

```
code .
```

Dentro de VS Code abre la terminal integrada desde el menú **Terminal → Nueva terminal**. A partir de ahora trabajarás en esa terminal; verifica que esté ubicada en la carpeta `Laboratorio_TDD`.

> **No necesitas `npm install`**
>
> El proyecto no tiene dependencias: todo lo que usamos viene incluido en Node.js.

Paso 4. Conoce los archivos

```
Laboratorio_TDD/
├── package.json          ← configuración y comando de pruebas
├── README.md
├── src/
│   └── carrito.js        ← código de producción (aquí va la lógica)
└── test/
    └── carrito.test.js   ← pruebas (aquí empiezas SIEMPRE)
```

`package.json` ya trae el comando `npm test`, que ejecuta `node --test`. La línea `"type": "module"` permite usar `import` y `export`.

package.json

```
{
  "name": "laboratorio-tdd-carrito",
  "version": "1.0.0",
  "description": "Laboratorio de TDD con JavaScript puro: carrito de compras",
  "type": "module",
  "scripts": {
    "test": "node --test",
    "test:watch": "node --test --watch"
  }
}
```

El código de producción empieza con una clase vacía:

src/carrito.js

```
// Laboratorio TDD: carrito de compras
// Escribe aquí el código SOLO después de tener una prueba en rojo.

export class Carrito {}
```

Y el archivo de pruebas ya importa las herramientas y crea un carrito nuevo antes de cada prueba:

test/carrito.test.js

```
import { describe, test, beforeEach } from 'node:test';
import assert from 'node:assert/strict';
import { Carrito } from '../src/carrito.js';

describe('Carrito de compras', () => {
  let carrito;

  beforeEach(() => {
    carrito = new Carrito();
  });

  // Escribe aquí tus pruebas, una iteración a la vez.
});
```

Paso 5. Ejecuta las pruebas por primera vez

```
npm test
```

✔ Resultado esperado: el proyecto funciona

TESTS	PASS	FAIL
0	0	0

Terminal

```
✔ Carrito de compras
ℹ tests 0
ℹ suites 1
ℹ pass 0
ℹ fail 0
```

Todavía no hay pruebas, por eso todo está en 0. Lo importante es que no aparezca ningún error. Si algo falla, revisa la sección **"Si algo no sale"** al final de la guía.

3. El reto: carrito de compras

Una tienda de tecnología en línea necesita la lógica de su carrito. El equipo de producto entregó esta historia de usuario:

> **Como** cliente de la tienda,
> **quiero** armar un carrito con productos, aplicar un cupón y ver el total con IVA,
> **para** saber exactamente cuánto voy a pagar antes de comprar.

Criterios de aceptación

#	Criterio
1	Un carrito nuevo está vacío: 0 ítems y subtotal $0.
2	Se pueden agregar productos con nombre, precio y cantidad (por defecto, 1).
3	Si se agrega un producto que ya está, se suma la cantidad en vez de duplicar la línea.
4	No se aceptan precios ≤ 0 ni cantidades que no sean enteros positivos.
5	Se pueden eliminar productos; eliminar uno inexistente es un error.
6	Cupones válidos: `DESC10` (10 %) y `DESC20` (20 %). Cualquier otro es un error.
7	El IVA es del 19 % y se calcula **después** del descuento. Total = subtotal − descuento + IVA.

> **Valores en pesos colombianos**
> Trabajamos en COP, sin decimales. Por eso redondeamos con `Math.round` los valores calculados con porcentajes.

La rutina de cada iteración

Cada criterio es una iteración. Todas siguen exactamente los mismos pasos:

1. **ROJO.** Pega la prueba en `test/carrito.test.js`.
2. **Ejecuta** `npm test` y comprueba que **falla** como indica la guía.
3. **VERDE.** Reemplaza el contenido de `src/carrito.js` por el código de la guía.
4. **Ejecuta** `npm test` y comprueba que **todo pasa**.
5. **REFACTOR.** Lee la reflexión y revisa si el código se puede mejorar.

> **Regla de control**
> No avances a la siguiente iteración hasta que tus números `tests`, `pass` y `fail` sean **idénticos** a los del recuadro **Resultado esperado**.

¿Dónde pego cada prueba?

Todas las iteraciones van **dentro** del `describe('Carrito de compras')`, una debajo de la otra. La última línea del archivo, `});`, siempre debe quedar al final:

test/carrito.test.js · estructura

```
describe('Carrito de compras', () => {
  let carrito;

  beforeEach(() => {
    carrito = new Carrito();
  });

  describe('Iteración 1: carrito vacío', () => { ... });

  describe('Iteración 2: agregar productos', () => { ... });

  // ← cada nueva iteración se pega aquí
});   // ← esta línea siempre queda al final
```

4. Laboratorio paso a paso

1

ITERACIÓN 1 DE 7

Carrito vacío

Requisito. Un carrito recién creado no tiene ítems y su subtotal es 0.

Paso 1 ROJO Escribe la prueba

Abre `test/carrito.test.js`. Selecciona la línea `// Escribe aquí tus pruebas, una iteración a la vez.` y **reemplázala** por este bloque:

test/carrito.test.js

```
describe('Iteración 1: carrito vacío', () => {
  test('un carrito nuevo no tiene ítems', () => {
    assert.equal(carrito.cantidadDeItems(), 0);
  });

  test('un carrito nuevo tiene subtotal 0', () => {
    assert.equal(carrito.subtotal(), 0);
  });
});
```

Paso 2 EJECUTAR Ejecuta y comprueba que FALLE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✖ Resultado esperado: PRUEBAS EN ROJO

TESTS	PASS	FAIL
2	0	2

Terminal

```
▶ Carrito de compras
  ▶ Iteración 1: carrito vacío
    ✖ un carrito nuevo no tiene ítems
    ✖ un carrito nuevo tiene subtotal 0
  ✖ Iteración 1: carrito vacío
✖ Carrito de compras
ℹ tests 2
ℹ pass 0
ℹ fail 2

✖ failing tests:

✖ un carrito nuevo no tiene ítems
  TypeError: carrito.cantidadDeItems is not a function

✖ un carrito nuevo tiene subtotal 0
  TypeError: carrito.subtotal is not a function
```

Fallan las 2 pruebas porque los métodos `cantidadDeItems()` y `subtotal()` todavía no existen.

Paso 3 VERDE Escribe el código mínimo

Abre `src/carrito.js`, selecciona todo (Ctrl + A) y **reemplázalo** por el siguiente código. Las líneas resaltadas en verde son las nuevas o modificadas respecto a la iteración anterior.

src/carrito.js · archivo completo — nuevo o modificado

```
// Laboratorio TDD: carrito de compras
// Escribe aquí el código SOLO después de tener una prueba en rojo.

export class Carrito {
  cantidadDeItems() {
    return 0;
  }

  subtotal() {
    return 0;
  }
}
```

Paso 4 EJECUTAR Ejecuta y comprueba que PASE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✔ **Resultado esperado: TODO EN VERDE**

TESTS	PASS	FAIL
2	2	0

Terminal

```
▶ Carrito de compras
  ▶ Iteración 1: carrito vacío
    ✔ un carrito nuevo no tiene ítems
    ✔ un carrito nuevo tiene subtotal 0
  ✔ Iteración 1: carrito vacío
✔ Carrito de compras
ℹ tests 2
ℹ pass 2
ℹ fail 0
```

Paso 5 REFACTOR Reflexiona: ¿hicimos trampa?

Sí, y a propósito. Devolver un valor fijo se llama **“fíngelo hasta lograrlo”** (*fake it till you make it*): es el código mínimo que pasa la prueba. La siguiente iteración nos obligará a escribir la lógica real; a esa técnica se le llama **triangulación**.

ITERACIÓN 2 DE 7

2 Agregar productos

Requisito. Al agregar productos, el carrito cuenta las unidades y calcula el subtotal (precio × cantidad). Si no se indica la cantidad, se asume 1.

Paso 1 ROJO Escribe la prueba

En `test/carrito.test.js`, pega este bloque **debajo del bloque de la Iteración 1** y **antes de la última línea** del archivo (`});`). Deja una línea en blanco entre bloques.

test/carrito.test.js

```
  describe('Iteración 2: agregar productos', () => {
    test('agregar un producto actualiza cantidad y subtotal', () => {
      carrito.agregar('Mouse', 50000, 2);

      assert.equal(carrito.cantidadDeItems(), 2);
      assert.equal(carrito.subtotal(), 100000);
    });

    test('agregar varios productos suma todos los subtotales', () => {
      carrito.agregar('Mouse', 50000, 2);
      carrito.agregar('Teclado', 120000, 1);

      assert.equal(carrito.cantidadDeItems(), 3);
      assert.equal(carrito.subtotal(), 220000);
    });

    test('si no se indica cantidad, se agrega una unidad', () => {
      carrito.agregar('Audífonos', 80000);

      assert.equal(carrito.cantidadDeItems(), 1);
    });
  });
```

Paso 2 EJECUTAR Ejecuta y comprueba que FALLE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✖ Resultado esperado: PRUEBAS EN ROJO

TESTS	PASS	FAIL
5	2	3

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 2: agregar productos
    ✖ agregar un producto actualiza cantidad y subtotal
    ✖ agregar varios productos suma todos los subtotales
    ✖ si no se indica cantidad, se agrega una unidad
  ✖ Iteración 2: agregar productos
✖ Carrito de compras
ℹ tests 5
ℹ pass 2
ℹ fail 3

✖ failing tests:

✖ agregar un producto actualiza cantidad y subtotal
  TypeError: carrito.agregar is not a function

✖ agregar varios productos suma todos los subtotales
  TypeError: carrito.agregar is not a function

✖ si no se indica cantidad, se agrega una unidad
  TypeError: carrito.agregar is not a function
```

Las 2 pruebas de la iteración 1 siguen pasando. Fallan las 3 nuevas porque el método `agregar()` no existe.

Paso 3 VERDE Escribe el código mínimo

El valor fijo ya no sirve: ahora necesitamos guardar los productos en un arreglo.

Abre `src/carrito.js`, selecciona todo (Ctrl + A) y **reemplázalo** por el siguiente código. Las líneas resaltadas en verde son las nuevas o modificadas respecto a la iteración anterior.

src/carrito.js · archivo completo — nuevo o modificado

```
// Laboratorio TDD: carrito de compras
// Escribe aquí el código SOLO después de tener una prueba en rojo.

export class Carrito {
  constructor() {
    this.productos = [];
  }

  agregar(nombre, precio, cantidad = 1) {
    this.productos.push({ nombre, precio, cantidad });
  }

  cantidadDeItems() {
    return this.productos.reduce((suma, p) => suma + p.cantidad, 0);
```

```
  }

  subtotal() {
    return this.productos.reduce((suma, p) => suma + p.precio * p.cantidad, 0);
  }
}
```

Paso 4 EJECUTAR Ejecuta y comprueba que PASE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✔ Resultado esperado: TODO EN VERDE

TESTS	PASS	FAIL
5	5	0

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 2: agregar productos
    ✔ agregar un producto actualiza cantidad y subtotal
    ✔ agregar varios productos suma todos los subtotales
    ✔ si no se indica cantidad, se agrega una unidad
  ✔ Iteración 2: agregar productos
✔ Carrito de compras
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

Paso 5 REFACTOR Revisa

¿Los nombres `suma` y `p` se entienden? Observa que las pruebas de la iteración 1 siguen en verde aunque cambiamos todo el código: esa es la **red de seguridad** que dan las pruebas.

ITERACIÓN 3 DE 7

3 Producto repetido

Requisito. Si se agrega un producto que ya está en el carrito, se acumula la cantidad en la misma línea.

Paso 1 ROJO Escribe la prueba

En `test/carrito.test.js`, pega este bloque **debajo del bloque de la Iteración 2** y **antes de la última línea** del archivo (`});`). Deja una línea en blanco entre bloques.

test/carrito.test.js

```
  describe('Iteración 3: producto repetido', () => {
    test('agregar el mismo producto acumula la cantidad en una sola línea', () => {
      carrito.agregar('Mouse', 50000, 1);
      carrito.agregar('Mouse', 50000, 2);

      assert.equal(carrito.productos.length, 1);
      assert.equal(carrito.cantidadDeItems(), 3);
      assert.equal(carrito.subtotal(), 150000);
    });
  });
```

Paso 2 EJECUTAR Ejecuta y comprueba que FALLE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✖ Resultado esperado: PRUEBAS EN ROJO

TESTS	PASS	FAIL
6	5	1

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 3: producto repetido
    ✖ agregar el mismo producto acumula la cantidad en una sola línea
  ✖ Iteración 3: producto repetido
✖ Carrito de compras
ℹ tests 6
ℹ pass 5
ℹ fail 1

✖ failing tests:

✖ agregar el mismo producto acumula la cantidad en una sola línea
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:
  2 !== 1
```

`2 !== 1` significa que el carrito tiene **2** líneas de "Mouse" y la prueba esperaba **1**.

Paso 3 VERDE Escribe el código mínimo

Solo cambia el método `agregar()`: antes de insertar, busca si el producto ya existe.

Abre `src/carrito.js`, selecciona todo (Ctrl + A) y **reemplázalo** por el siguiente código. Las líneas resaltadas en verde son las nuevas o modificadas respecto a la iteración anterior.

src/carrito.js · archivo completo — nuevo o modificado

```
// Laboratorio TDD: carrito de compras
// Escribe aquí el código SOLO después de tener una prueba en rojo.

export class Carrito {
  constructor() {
    this.productos = [];
  }

  agregar(nombre, precio, cantidad = 1) {
    const existente = this.productos.find((p) => p.nombre === nombre);

    if (existente) {
      existente.cantidad += cantidad;
    } else {
      this.productos.push({ nombre, precio, cantidad });
    }
  }

  cantidadDeItems() {
    return this.productos.reduce((suma, p) => suma + p.cantidad, 0);
  }
```

```
  subtotal() {
    return this.productos.reduce((suma, p) => suma + p.precio * p.cantidad, 0);
  }
}
```

Paso 4 EJECUTAR Ejecuta y comprueba que PASE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✔ Resultado esperado: TODO EN VERDE

TESTS	PASS	FAIL
6	6	0

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 3: producto repetido
    ✔ agregar el mismo producto acumula la cantidad en una sola línea
  ✔ Iteración 3: producto repetido
✔ Carrito de compras
ℹ tests 6
ℹ pass 6
ℹ fail 0
```

Paso 5 REFACTOR Observa

Modificamos código que ya funcionaba y las 5 pruebas anteriores nos confirmaron que no rompimos nada.

ITERACIÓN 4 DE 7

4 Validaciones

Requisito. No se aceptan precios menores o iguales a 0, ni cantidades que no sean enteros positivos.

Paso 1 ROJO Escribe la prueba

En `test/carrito.test.js`, pega este bloque **debajo del bloque de la Iteración 3** y **antes de la última línea** del archivo (`});`). Deja una línea en blanco entre bloques.

test/carrito.test.js

```
  describe('Iteración 4: validaciones', () => {
    test('no permite precios negativos o en cero', () => {
      assert.throws(() => carrito.agregar('Mouse', -100, 1), /precio/);
      assert.throws(() => carrito.agregar('Mouse', 0, 1), /precio/);
    });

    test('no permite cantidades en cero, negativas o decimales', () => {
      assert.throws(() => carrito.agregar('Mouse', 50000, 0), /cantidad/);
      assert.throws(() => carrito.agregar('Mouse', 50000, -1), /cantidad/);
      assert.throws(() => carrito.agregar('Mouse', 50000, 1.5), /cantidad/);
    });
  });
```

Paso 2 EJECUTAR Ejecuta y comprueba que FALLE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✖ Resultado esperado: PRUEBAS EN ROJO

TESTS	PASS	FAIL
8	6	2

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 4: validaciones
    ✖ no permite precios negativos o en cero
    ✖ no permite cantidades en cero, negativas o decimales
  ✖ Iteración 4: validaciones
✖ Carrito de compras
ℹ tests 8
ℹ pass 6
ℹ fail 2

✖ failing tests:

✖ no permite precios negativos o en cero
  AssertionError [ERR_ASSERTION]: Missing expected exception.

✖ no permite cantidades en cero, negativas o decimales
  AssertionError [ERR_ASSERTION]: Missing expected exception.
```

`Missing expected exception` significa que la prueba esperaba un error, pero `agregar()` aceptó los valores inválidos sin quejarse.

Paso 3 VERDE Escribe el código mínimo

Abre `src/carrito.js`, selecciona todo (Ctrl + A) y **reemplázalo** por el siguiente código. Las líneas resaltadas en verde son las nuevas o modificadas respecto a la iteración anterior.

src/carrito.js · archivo completo — nuevo o modificado

```
// Laboratorio TDD: carrito de compras
// Escribe aquí el código SOLO después de tener una prueba en rojo.

export class Carrito {
  constructor() {
    this.productos = [];
  }

  agregar(nombre, precio, cantidad = 1) {
    if (typeof precio !== 'number' || precio <= 0) {
      throw new Error('El precio debe ser mayor que cero');
    }
    if (!Number.isInteger(cantidad) || cantidad <= 0) {
      throw new Error('La cantidad debe ser un entero positivo');
    }

    const existente = this.productos.find((p) => p.nombre === nombre);

    if (existente) {
```

```
      existente.cantidad += cantidad;
    } else {
      this.productos.push({ nombre, precio, cantidad });
    }
  }

  cantidadDeItems() {
    return this.productos.reduce((suma, p) => suma + p.cantidad, 0);
  }

  subtotal() {
    return this.productos.reduce((suma, p) => suma + p.precio * p.cantidad, 0);
  }
}
```

Paso 4 EJECUTAR Ejecuta y comprueba que PASE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✔ Resultado esperado: TODO EN VERDE

TESTS	PASS	FAIL
8	8	0

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 4: validaciones
    ✔ no permite precios negativos o en cero
    ✔ no permite cantidades en cero, negativas o decimales
  ✔ Iteración 4: validaciones
✔ Carrito de compras
ℹ tests 8
ℹ pass 8
ℹ fail 0
```

Paso 5 REFACTOR Piensa en los límites

Probamos **los bordes**: 0, −1 y 1.5. Los errores suelen esconderse justo en los límites. ¿Qué otro caso agregarías? (Pista: ¿y si el precio llega como texto, `'50000'`?)

ITERACIÓN 5 DE 7

5 Eliminar productos

Requisito. Se puede quitar un producto del carrito por su nombre. Si el producto no existe, se lanza un error.

Paso 1 ROJO Escribe la prueba

En `test/carrito.test.js`, pega este bloque **debajo del bloque de la Iteración 4** y **antes de la última línea** del archivo (`});`). Deja una línea en blanco entre bloques.

test/carrito.test.js

```
  describe('Iteración 5: eliminar productos', () => {
    test('eliminar un producto lo quita del carrito', () => {
      carrito.agregar('Mouse', 50000, 2);
      carrito.agregar('Teclado', 120000, 1);

      carrito.eliminar('Mouse');

      assert.equal(carrito.cantidadDeItems(), 1);
      assert.equal(carrito.subtotal(), 120000);
    });

    test('eliminar un producto que no existe lanza un error', () => {
      assert.throws(() => carrito.eliminar('Monitor'), /no está en el carrito/);
    });
  });
```

Paso 2 EJECUTAR Ejecuta y comprueba que FALLE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✖ Resultado esperado: PRUEBAS EN ROJO

TESTS	PASS	FAIL
10	8	2

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 5: eliminar productos
    ✖ eliminar un producto lo quita del carrito
    ✖ eliminar un producto que no existe lanza un error
  ✖ Iteración 5: eliminar productos
✖ Carrito de compras
ℹ tests 10
ℹ pass 8
ℹ fail 2

✖ failing tests:

✖ eliminar un producto lo quita del carrito
  TypeError: carrito.eliminar is not a function

✖ eliminar un producto que no existe lanza un error
  AssertionError [ERR_ASSERTION]: The input did not match the regular expression /no está en
el carrito/. Input:
  'TypeError: carrito.eliminar is not a function'
```

El segundo mensaje es largo, pero dice algo simple: se esperaba un error con el texto `no está en el carrito` y en su lugar llegó `eliminar is not a function`, porque el método aún no existe.

Paso 3 VERDE Escribe el código mínimo

Abre `src/carrito.js`, selecciona todo (Ctrl + A) y **reemplázalo** por el siguiente código. Las líneas resaltadas en verde son las nuevas o modificadas respecto a la iteración anterior.

src/carrito.js · archivo completo — nuevo o modificado

```
// Laboratorio TDD: carrito de compras
// Escribe aquí el código SOLO después de tener una prueba en rojo.

export class Carrito {
  constructor() {
    this.productos = [];
  }

  agregar(nombre, precio, cantidad = 1) {
    if (typeof precio !== 'number' || precio <= 0) {
      throw new Error('El precio debe ser mayor que cero');
    }
    if (!Number.isInteger(cantidad) || cantidad <= 0) {
      throw new Error('La cantidad debe ser un entero positivo');
    }

    const existente = this.productos.find((p) => p.nombre === nombre);
```

```
    if (existente) {
      existente.cantidad += cantidad;
    } else {
      this.productos.push({ nombre, precio, cantidad });
    }
  }

  eliminar(nombre) {
    const indice = this.productos.findIndex((p) => p.nombre === nombre);

    if (indice === -1) {
      throw new Error(`El producto "${nombre}" no está en el carrito`);
    }
    this.productos.splice(indice, 1);
  }

  cantidadDeItems() {
    return this.productos.reduce((suma, p) => suma + p.cantidad, 0);
  }

  subtotal() {
    return this.productos.reduce((suma, p) => suma + p.precio * p.cantidad, 0);
  }
}
```

Paso 4 EJECUTAR Ejecuta y comprueba que PASE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✔ Resultado esperado: TODO EN VERDE

TESTS	PASS	FAIL
10	10	0

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 5: eliminar productos
    ✔ eliminar un producto lo quita del carrito
    ✔ eliminar un producto que no existe lanza un error
  ✔ Iteración 5: eliminar productos
✔ Carrito de compras
ℹ tests 10
ℹ pass 10
ℹ fail 0
```

Paso 5 REFACTOR Revisa

¿El mensaje de error le dice claramente al usuario qué pasó? Un buen mensaje incluye el dato que causó el problema.

ITERACIÓN 6 DE 7

6 Cupones de descuento

Requisito. Los cupones `DESC10` y `DESC20` descuentan el 10 % y el 20 % del subtotal. Un cupón desconocido es un error.

Paso 1 ROJO Escribe la prueba

En `test/carrito.test.js`, pega este bloque **debajo del bloque de la Iteración 5** y **antes de la última línea** del archivo (`});`). Deja una línea en blanco entre bloques.

test/carrito.test.js

```
describe('Iteración 6: cupones de descuento', () => {
  test('sin cupón el descuento es 0', () => {
    carrito.agregar('Teclado', 200000, 1);
    assert.equal(carrito.descuento(), 0);
  });

  test('el cupón DESC10 descuenta el 10 % del subtotal', () => {
    carrito.agregar('Teclado', 200000, 1);
    carrito.aplicarCupon('DESC10');
    assert.equal(carrito.descuento(), 20000);
  });

  test('el cupón DESC20 descuenta el 20 % del subtotal', () => {
    carrito.agregar('Teclado', 200000, 1);
    carrito.aplicarCupon('DESC20');
    assert.equal(carrito.descuento(), 40000);
  });

  test('un cupón inexistente lanza un error', () => {
    assert.throws(() => carrito.aplicarCupon('GRATIS'), /no es válido/);
  });
});
```

Paso 2 EJECUTAR Ejecuta y comprueba que FALLE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✖ **Resultado esperado: PRUEBAS EN ROJO**

TESTS	PASS	FAIL
14	10	4

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 6: cupones de descuento
    ✖ sin cupón el descuento es 0
    ✖ el cupón DESC10 descuenta el 10 % del subtotal
    ✖ el cupón DESC20 descuenta el 20 % del subtotal
    ✖ un cupón inexistente lanza un error
  ✖ Iteración 6: cupones de descuento
✖ Carrito de compras
ℹ tests 14
ℹ pass 10
ℹ fail 4

✖ failing tests:

✖ sin cupón el descuento es 0
  TypeError: carrito.descuento is not a function

✖ el cupón DESC10 descuenta el 10 % del subtotal
  TypeError: carrito.aplicarCupon is not a function

✖ el cupón DESC20 descuenta el 20 % del subtotal
  TypeError: carrito.aplicarCupon is not a function

✖ un cupón inexistente lanza un error
  AssertionError [ERR_ASSERTION]: The input did not match the regular expression /no es
válido/. Input:
  'TypeError: carrito.aplicarCupon is not a function'
```

Fallan las 4 pruebas nuevas porque `aplicarCupon()` y `descuento()` no existen. Las 10 anteriores siguen en verde.

Paso 3 VERDE Escribe el código mínimo

Guardamos los cupones en un objeto `CUPONES` fuera de la clase. Así, agregar un cupón nuevo es una línea más y no un `if` más.

Abre `src/carrito.js`, selecciona todo (Ctrl + A) y **reemplázalo** por el siguiente código. Las líneas resaltadas en verde son las nuevas o modificadas respecto a la iteración anterior.

src/carrito.js · archivo completo — nuevo o modificado

```
// Laboratorio TDD: carrito de compras
// Escribe aquí el código SOLO después de tener una prueba en rojo.

const CUPONES = {
  DESC10: 0.10,
  DESC20: 0.20,
```

```
};

export class Carrito {
  constructor() {
    this.productos = [];
    this.cupon = null;
  }

  agregar(nombre, precio, cantidad = 1) {
    if (typeof precio !== 'number' || precio <= 0) {
      throw new Error('El precio debe ser mayor que cero');
    }
    if (!Number.isInteger(cantidad) || cantidad <= 0) {
      throw new Error('La cantidad debe ser un entero positivo');
    }

    const existente = this.productos.find((p) => p.nombre === nombre);

    if (existente) {
      existente.cantidad += cantidad;
    } else {
      this.productos.push({ nombre, precio, cantidad });
    }
  }

  eliminar(nombre) {
    const indice = this.productos.findIndex((p) => p.nombre === nombre);

    if (indice === -1) {
      throw new Error(`El producto "${nombre}" no está en el carrito`);
    }
    this.productos.splice(indice, 1);
  }

  cantidadDeItems() {
    return this.productos.reduce((suma, p) => suma + p.cantidad, 0);
  }

  subtotal() {
    return this.productos.reduce((suma, p) => suma + p.precio * p.cantidad, 0);
  }

  aplicarCupon(codigo) {
    if (!(codigo in CUPONES)) {
      throw new Error(`El cupón "${codigo}" no es válido`);
    }
    this.cupon = codigo;
  }

  descuento() {
    if (!this.cupon) return 0;
    return Math.round(this.subtotal() * CUPONES[this.cupon]);
  }
}
```

Paso 4 EJECUTAR Ejecuta y comprueba que PASE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✔ Resultado esperado: TODO EN VERDE

TESTS	PASS	FAIL
14	14	0

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 6: cupones de descuento
    ✔ sin cupón el descuento es 0
    ✔ el cupón DESC10 descuenta el 10 % del subtotal
    ✔ el cupón DESC20 descuenta el 20 % del subtotal
    ✔ un cupón inexistente lanza un error
  ✔ Iteración 6: cupones de descuento
✔ Carrito de compras
ℹ tests 14
ℹ pass 14
ℹ fail 0
```

Paso 5 REFACTOR Revisa

¿Qué pasaría si mañana la tienda crea el cupón `DESC30`? Solo agregarías una línea en `CUPONES` y una prueba nueva.

ITERACIÓN 7 DE 7

7 IVA y total a pagar

Requisito. El IVA es del 19 % y se calcula sobre el subtotal **menos** el descuento. El total es esa base más el IVA.

Ejemplo con cupón DESC10	Valor
Subtotal (2 × $50.000)	$100.000
Descuento 10 %	− $10.000
Base gravable	$90.000
IVA 19 %	+ $17.100
Total a pagar	**$107.100**

Paso 1 ROJO Escribe la prueba

En `test/carrito.test.js`, pega este bloque **debajo del bloque de la Iteración 6** y **antes de la última línea** del archivo (`});`). Deja una línea en blanco entre bloques.

test/carrito.test.js

```
  describe('Iteración 7: IVA y total a pagar', () => {
    test('el IVA es el 19 % del subtotal cuando no hay cupón', () => {
      carrito.agregar('Mouse', 50000, 2);

      assert.equal(carrito.iva(), 19000);
      assert.equal(carrito.total(), 119000);
    });

    test('el IVA se calcula sobre el valor con descuento', () => {
      carrito.agregar('Mouse', 50000, 2);
      carrito.aplicarCupon('DESC10');

      assert.equal(carrito.iva(), 17100);
      assert.equal(carrito.total(), 107100);
    });

    test('un carrito vacío tiene total 0', () => {
      assert.equal(carrito.total(), 0);
    });
  });
```

Paso 2 EJECUTAR Ejecuta y comprueba que FALLE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✖ Resultado esperado: PRUEBAS EN ROJO

TESTS	PASS	FAIL
17	14	3

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 7: IVA y total a pagar
    ✖ el IVA es el 19 % del subtotal cuando no hay cupón
    ✖ el IVA se calcula sobre el valor con descuento
    ✖ un carrito vacío tiene total 0
  ✖ Iteración 7: IVA y total a pagar
✖ Carrito de compras
ℹ tests 17
ℹ pass 14
ℹ fail 3

✖ failing tests:

✖ el IVA es el 19 % del subtotal cuando no hay cupón
  TypeError: carrito.iva is not a function

✖ el IVA se calcula sobre el valor con descuento
  TypeError: carrito.iva is not a function

✖ un carrito vacío tiene total 0
  TypeError: carrito.total is not a function
```

Fallan las 3 pruebas nuevas porque `iva()` y `total()` no existen.

Paso 3 VERDE Escribe el código mínimo

Abre `src/carrito.js`, selecciona todo (Ctrl + A) y **reemplázalo** por el siguiente código. Las líneas resaltadas en verde son las nuevas o modificadas respecto a la iteración anterior.

src/carrito.js · archivo completo — nuevo o modificado

```
// Laboratorio TDD: carrito de compras
// Escribe aquí el código SOLO después de tener una prueba en rojo.

const IVA = 0.19;

const CUPONES = {
  DESC10: 0.10,
  DESC20: 0.20,
};

export class Carrito {
  constructor() {
    this.productos = [];
    this.cupon = null;
```

```
  }

  agregar(nombre, precio, cantidad = 1) {
    if (typeof precio !== 'number' || precio <= 0) {
      throw new Error('El precio debe ser mayor que cero');
    }
    if (!Number.isInteger(cantidad) || cantidad <= 0) {
      throw new Error('La cantidad debe ser un entero positivo');
    }

    const existente = this.productos.find((p) => p.nombre === nombre);

    if (existente) {
      existente.cantidad += cantidad;
    } else {
      this.productos.push({ nombre, precio, cantidad });
    }
  }

  eliminar(nombre) {
    const indice = this.productos.findIndex((p) => p.nombre === nombre);

    if (indice === -1) {
      throw new Error(`El producto "${nombre}" no está en el carrito`);
    }
    this.productos.splice(indice, 1);
  }

  cantidadDeItems() {
    return this.productos.reduce((suma, p) => suma + p.cantidad, 0);
  }

  subtotal() {
    return this.productos.reduce((suma, p) => suma + p.precio * p.cantidad, 0);
  }

  aplicarCupon(codigo) {
    if (!(codigo in CUPONES)) {
      throw new Error(`El cupón "${codigo}" no es válido`);
    }
    this.cupon = codigo;
  }

  descuento() {
    if (!this.cupon) return 0;
    return Math.round(this.subtotal() * CUPONES[this.cupon]);
  }

  iva() {
    return Math.round((this.subtotal() - this.descuento()) * IVA);
  }

  total() {
    return this.subtotal() - this.descuento() + this.iva();
  }
```

```
}
```

Paso 4 EJECUTAR Ejecuta y comprueba que PASE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✔ Resultado esperado: TODO EN VERDE

TESTS	PASS	FAIL
17	17	0

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 7: IVA y total a pagar
    ✔ el IVA es el 19 % del subtotal cuando no hay cupón
    ✔ el IVA se calcula sobre el valor con descuento
    ✔ un carrito vacío tiene total 0
  ✔ Iteración 7: IVA y total a pagar
✔ Carrito de compras
ℹ tests 17
ℹ pass 17
ℹ fail 0
```

Paso 5 REFACTOR Elimina la duplicación

La expresión `this.subtotal() - this.descuento()` aparece **dos veces**. Vamos a extraerla a un método con un nombre que explique qué es: `baseGravable()`. Las pruebas **no cambian**. Reemplaza de nuevo todo el contenido de `src/carrito.js`:

src/carrito.js · archivo completo (refactor) — nuevo o modificado

```
// Laboratorio TDD: carrito de compras
// Escribe aquí el código SOLO después de tener una prueba en rojo.

const IVA = 0.19;

const CUPONES = {
  DESC10: 0.10,
  DESC20: 0.20,
};

export class Carrito {
  constructor() {
    this.productos = [];
    this.cupon = null;
  }

  agregar(nombre, precio, cantidad = 1) {
    if (typeof precio !== 'number' || precio <= 0) {
      throw new Error('El precio debe ser mayor que cero');
```

```
    }
    if (!Number.isInteger(cantidad) || cantidad <= 0) {
      throw new Error('La cantidad debe ser un entero positivo');
    }

    const existente = this.productos.find((p) => p.nombre === nombre);

    if (existente) {
      existente.cantidad += cantidad;
    } else {
      this.productos.push({ nombre, precio, cantidad });
    }
  }

  eliminar(nombre) {
    const indice = this.productos.findIndex((p) => p.nombre === nombre);

    if (indice === -1) {
      throw new Error(`El producto "${nombre}" no está en el carrito`);
    }
    this.productos.splice(indice, 1);
  }

  cantidadDeItems() {
    return this.productos.reduce((suma, p) => suma + p.cantidad, 0);
  }

  subtotal() {
    return this.productos.reduce((suma, p) => suma + p.precio * p.cantidad, 0);
  }

  aplicarCupon(codigo) {
    if (!(codigo in CUPONES)) {
      throw new Error(`El cupón "${codigo}" no es válido`);
    }
    this.cupon = codigo;
  }

  descuento() {
    if (!this.cupon) return 0;
    return Math.round(this.subtotal() * CUPONES[this.cupon]);
  }

  baseGravable() {
    return this.subtotal() - this.descuento();
  }

  iva() {
    return Math.round(this.baseGravable() * IVA);
  }

  total() {
    return this.baseGravable() + this.iva();
  }
}
```

Paso 6 EJECUTAR Ejecuta y comprueba que TODO SIGA EN VERDE

Guarda los cambios (Ctrl + S) y en la terminal ejecuta:

```
npm test
```

✔ Resultado esperado: TODO EN VERDE

TESTS	PASS	FAIL
17	17	0

Terminal

```
▶ Carrito de compras
  …  (iteraciones anteriores, todas con ✔)
  ▶ Iteración 7: IVA y total a pagar
    ✔ el IVA es el 19 % del subtotal cuando no hay cupón
    ✔ el IVA se calcula sobre el valor con descuento
    ✔ un carrito vacío tiene total 0
  ✔ Iteración 7: IVA y total a pagar
✔ Carrito de compras
ℹ tests 17
ℹ pass 17
ℹ fail 0
```

Mismo resultado que antes del refactor: cambiamos la estructura del código sin cambiar su comportamiento. Eso es refactorizar.

5. Resultado final

Si completaste todas las iteraciones, al ejecutar `npm test` tu terminal debe mostrar **exactamente** esto:

✔ Laboratorio completo

TESTS	PASS	FAIL
17	17	0

Terminal

```
▶ Carrito de compras
  ▶ Iteración 1: carrito vacío
    ✔ un carrito nuevo no tiene ítems
    ✔ un carrito nuevo tiene subtotal 0
  ✔ Iteración 1: carrito vacío
  ▶ Iteración 2: agregar productos
    ✔ agregar un producto actualiza cantidad y subtotal
    ✔ agregar varios productos suma todos los subtotales
    ✔ si no se indica cantidad, se agrega una unidad
  ✔ Iteración 2: agregar productos
  ▶ Iteración 3: producto repetido
    ✔ agregar el mismo producto acumula la cantidad en una sola línea
  ✔ Iteración 3: producto repetido
  ▶ Iteración 4: validaciones
    ✔ no permite precios negativos o en cero
    ✔ no permite cantidades en cero, negativas o decimales
  ✔ Iteración 4: validaciones
  ▶ Iteración 5: eliminar productos
    ✔ eliminar un producto lo quita del carrito
    ✔ eliminar un producto que no existe lanza un error
  ✔ Iteración 5: eliminar productos
  ▶ Iteración 6: cupones de descuento
    ✔ sin cupón el descuento es 0
    ✔ el cupón DESC10 descuenta el 10 % del subtotal
    ✔ el cupón DESC20 descuenta el 20 % del subtotal
    ✔ un cupón inexistente lanza un error
  ✔ Iteración 6: cupones de descuento
  ▶ Iteración 7: IVA y total a pagar
    ✔ el IVA es el 19 % del subtotal cuando no hay cupón
    ✔ el IVA se calcula sobre el valor con descuento
    ✔ un carrito vacío tiene total 0
  ✔ Iteración 7: IVA y total a pagar
✔ Carrito de compras
ℹ tests 17
ℹ suites 8
ℹ pass 17
ℹ fail 0
```

Los tiempos en milisegundos que muestra la terminal al lado de cada prueba varían en cada equipo; no hacen parte del resultado.

Si algo no sale

Lo que ves	Causa probable y solución
`'node' no se reconoce como un comando`	Node.js no está instalado o la terminal se abrió antes de instalarlo. Instálalo y abre una terminal nueva.
`npm error Missing script: "test"` o `ENOENT ... package.json`	La terminal no está en la carpeta del proyecto. Ejecuta `cd Laboratorio_TDD`.
`SyntaxError: Unexpected token` o `Unexpected end of input`	El bloque se pegó fuera de lugar o falta/sobra un `});`. Revisa la estructura de la sección 3: el archivo debe terminar con **un solo** `});`.
Los números no cambian después de pegar	No guardaste el archivo. Presiona Ctrl + S y vuelve a ejecutar `npm test`.
`tests` es mayor o menor de lo esperado	Pegaste un bloque dos veces o te faltó uno. Cada iteración se pega **una sola vez**.
`is not a function` en el paso VERDE	El código de `src/carrito.js` no quedó completo. Selecciona todo y vuelve a pegar el archivo completo de la iteración.
Ves líneas como `ok 1 - ...` y `# pass 17`	Es el mismo resultado en otro formato (TAP). Compara los números de `# tests`, `# pass` y `# fail`.

6. Retos adicionales

Resuélvelos con el mismo ciclo: primero la prueba, luego el código. Aquí no hay solución; ahí está la gracia.

A **Resumen de la compra.** Crea `resumen()`, que devuelva un objeto con `subtotal`, `descuento`, `iva` y `total`. Te dejamos la primera prueba:

```
test('resumen devuelve todos los valores de la compra', () => {
  carrito.agregar('Mouse', 50000, 2);
  carrito.aplicarCupon('DESC10');

  assert.deepEqual(carrito.resumen(), {
    subtotal: 100000,
    descuento: 10000,
    iva: 17100,
    total: 107100,
  });
});
```

B **Costo de envío.** El envío cuesta $12.000, pero es **gratis** si la base gravable es de $150.000 o más. Agrega `envio()` e inclúyelo en el total (el envío no paga IVA). Prueba el caso límite: exactamente $150.000.

C **Actualizar cantidad.** Crea `actualizarCantidad(nombre, nueva)`. Si la nueva cantidad es 0, el producto se elimina. Si es negativa, se lanza un error.

D **Cupones sin importar mayúsculas.** `'desc10'` y `' DESC10 '` (con espacios) también deben funcionar.

E **Cupón de valor fijo.** El cupón `MENOS30K` resta $30.000, pero el descuento **nunca** puede dejar la base en negativo. ¿Qué cambia en la estructura de `CUPONES`? Refactoriza con las pruebas en verde.

7. Errores comunes al hacer TDD

Error	Por qué es un problema
Escribir varias pruebas de una vez	Pierdes el ritmo del ciclo y no sabes cuál cambio arregló qué.
No ver la prueba en rojo	Una prueba que nunca falló puede estar mal escrita y pasar siempre.
Escribir más código del necesario	Ese código extra no tiene prueba que lo respalde.
Pruebas que dependen unas de otras	Si una falla, arrastra a las demás. Por eso usamos `beforeEach`.
Refactorizar con pruebas en rojo	No sabrás si el fallo viene del refactor o de antes.
Nombres de prueba vagos (`'test 1'`)	El nombre debe describir el comportamiento: es documentación.

8. Cierre

Preguntas para la reflexión

1. ¿En qué momento las pruebas te avisaron de que rompiste algo que ya funcionaba?
2. ¿Cómo cambió tu forma de pensar el problema al escribir la prueba primero?
3. Si mañana el IVA cambia al 16 %, ¿cuántas líneas tendrías que tocar y cómo sabrías que todo sigue bien?
4. ¿Qué parte del carrito te costaría más probar si no la hubieras diseñado con TDD?

Lista de chequeo

- [ ] Cloné el repositorio y `npm test` funcionó sin instalar dependencias.
- [ ] Vi fallar cada iteración (rojo) antes de hacerla pasar (verde).
- [ ] Mis números coincidieron con cada recuadro de **Resultado esperado**.
- [ ] Terminé con **17 pruebas, 17 en verde y 0 fallos**.
- [ ] Extraje `baseGravable()` sin romper ninguna prueba.
- [ ] Resolví al menos un reto adicional con el ciclo completo.